鸿蒙开发流程

1、鸿蒙开发背景介绍

2、基础开发工具介绍

3、Demo 开发演示

4、本地打包流程

4.1、本地打包流程配置

4.2、打包输出文件介绍

5、线上发布流程

5.1、线上发布流程配置

5.2、线上发布注意事项

鸿蒙开发背景介绍

大背景：大国制裁，备胎转正

小背景详情见鸿蒙白皮书

开发的产物

1、传统的app

2、元服务：系统级别卡片式的应用，类似于的小程序

技术理念

- 一次开发，多端部署

> 开发者，只需要基于统一的 鸿蒙API 规范，一套工程进行开发，由于鸿蒙系统，底层对于不同的设备兼容性进行处理，就可以做到，一套工程开发，打包出适应不同设备的app， app 上架分发的时候，就可以根据不同的设备进行分发。

- 可分可合，自由流转

> 在开发时，把不同的业务解耦成不同的模块，在部署时，将一个或者多个模块自由组合或拆分，打包App pack上架。
>
> 基于同一套的鸿蒙 API ，开发者可以开发一套应用，在不同的设备中部署安装，自由流转

- 统一生态，原生智能

> 对于不同操作系统的渲染模式（webView 渲染、原生渲染、自渲染），鸿蒙分别提供 WebView、ArkUI和XComponent 三种不同的方式进行底层支持。同时主流的框架平台，逐渐适配鸿蒙OS。
>
> 鸿蒙OS 通过提供不同 AI 套件的形式，提供原生支持 AI 的能力。

✧ MachineLearning Kit：提供场景化能力，包括通用卡证识别、实时语音识别等；提供 AI 控件能力，使系统控件融合文字识别等 AI 能力。

✧ Core AI API：提供图像语义、语言语音解析、OCR 文字识别等能力。

✧ Core DeepLearning API：提供高性能低功耗的端侧推理和端侧学习环境。

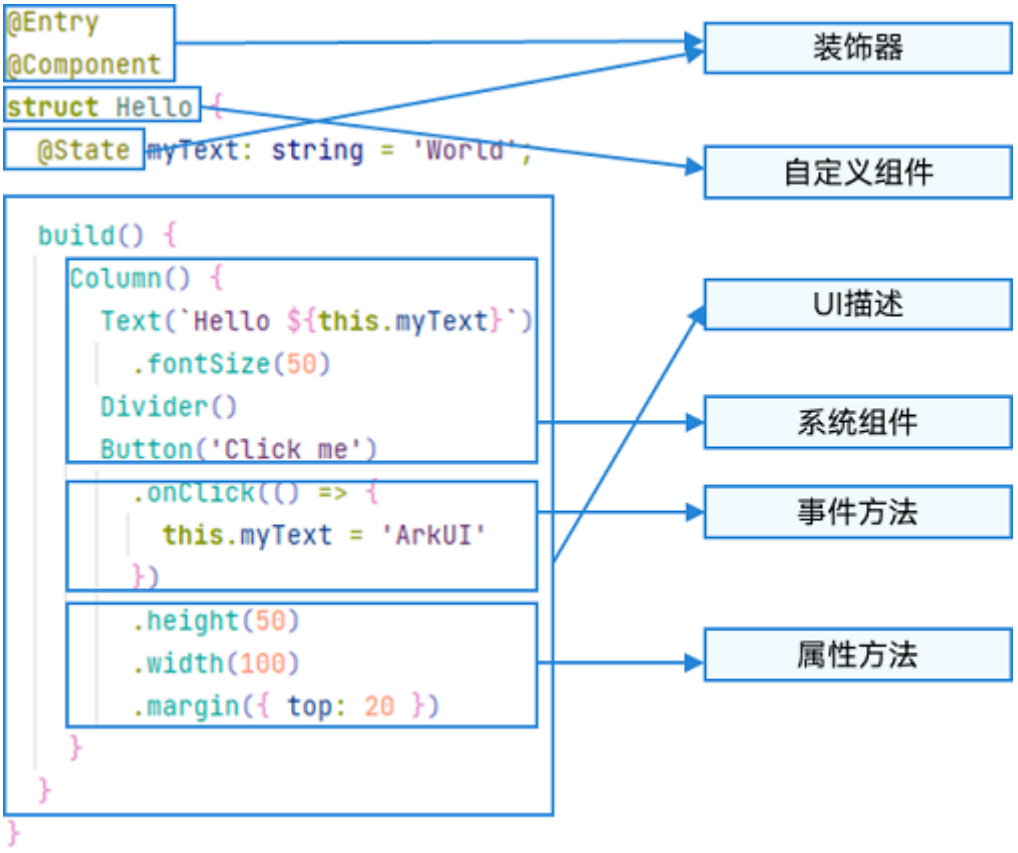
基础开发工具介绍

工具介绍

- 开发工具 DevEco：集成开发、调试、模拟器等功能。下载需要注册华为账号。
- UI 框架 ArkUI：声明式的UI 框架，组件有哪些属性和样式，从写法上一目了然
- 开发语言 ArkTs：鸿蒙应用开发语言，在保持 TS 语法的基础上，提供了更加强的约束。

UI 框架 ArkUI & 开发语言 ArkTs 介绍

ArkUI & ArkTs API 文档

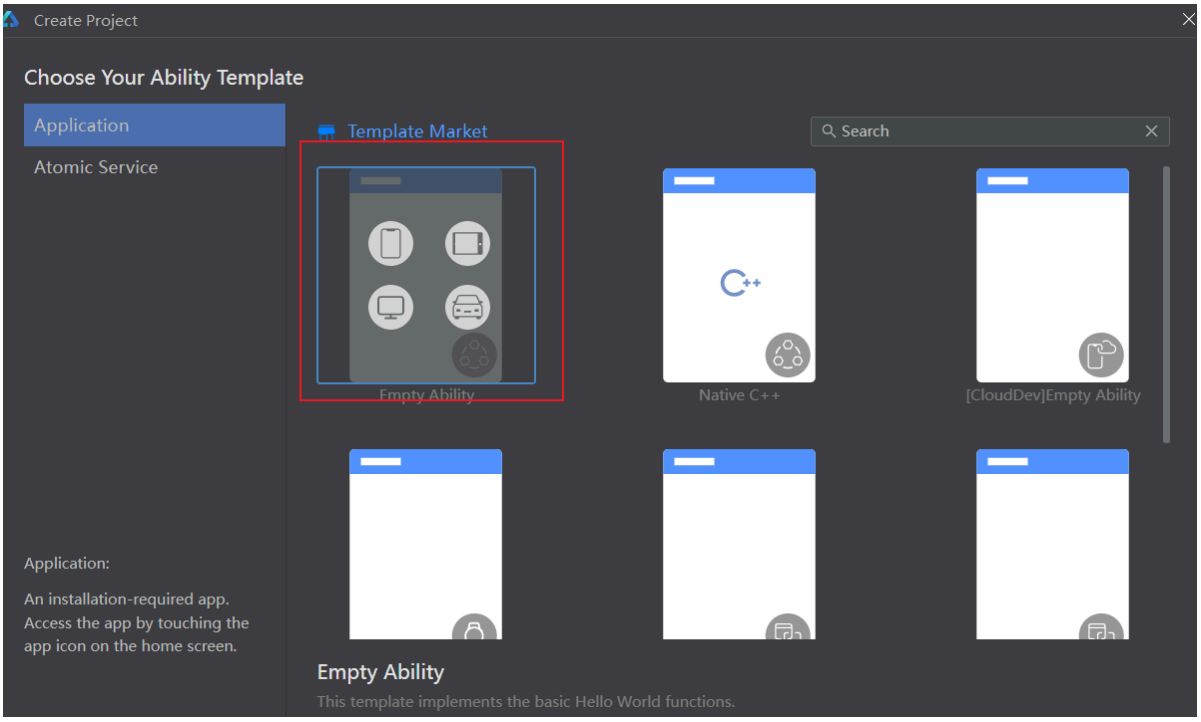


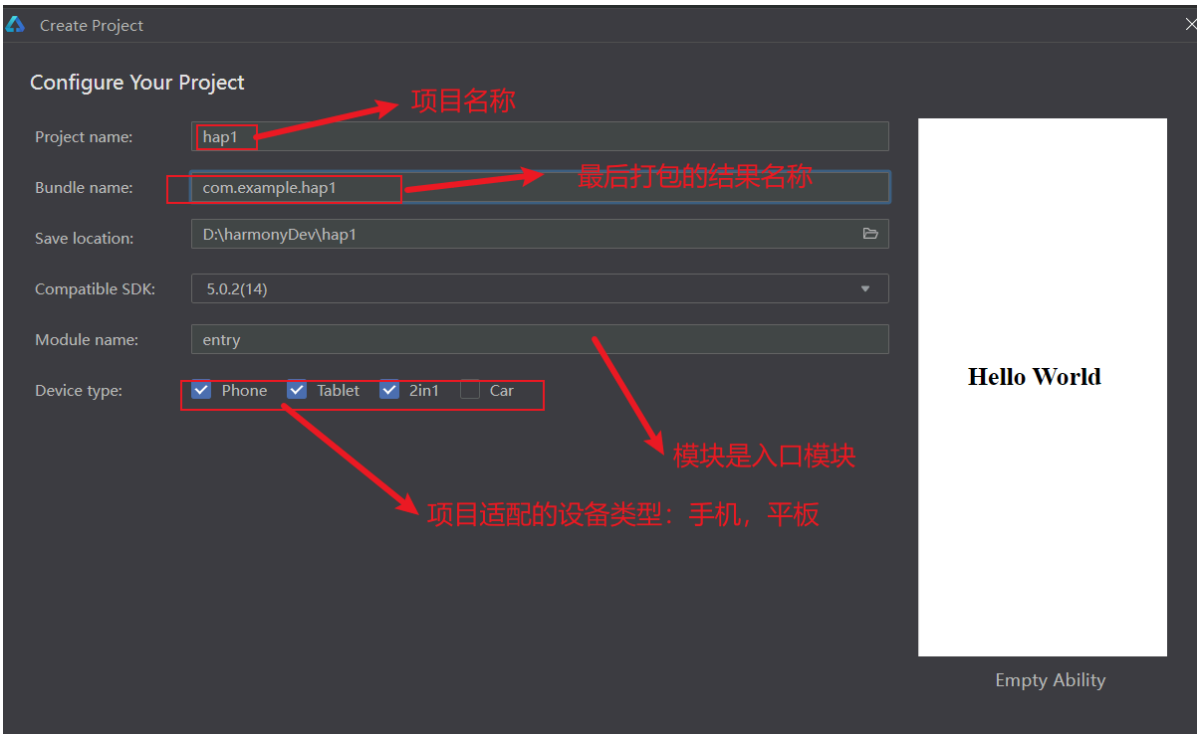
Demo 开发演示

项目创建

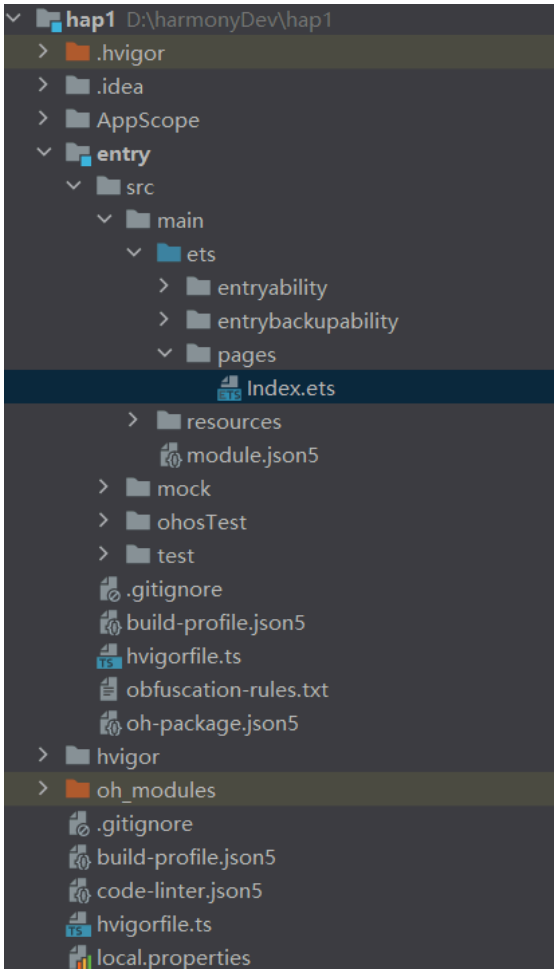
- 创建步骤







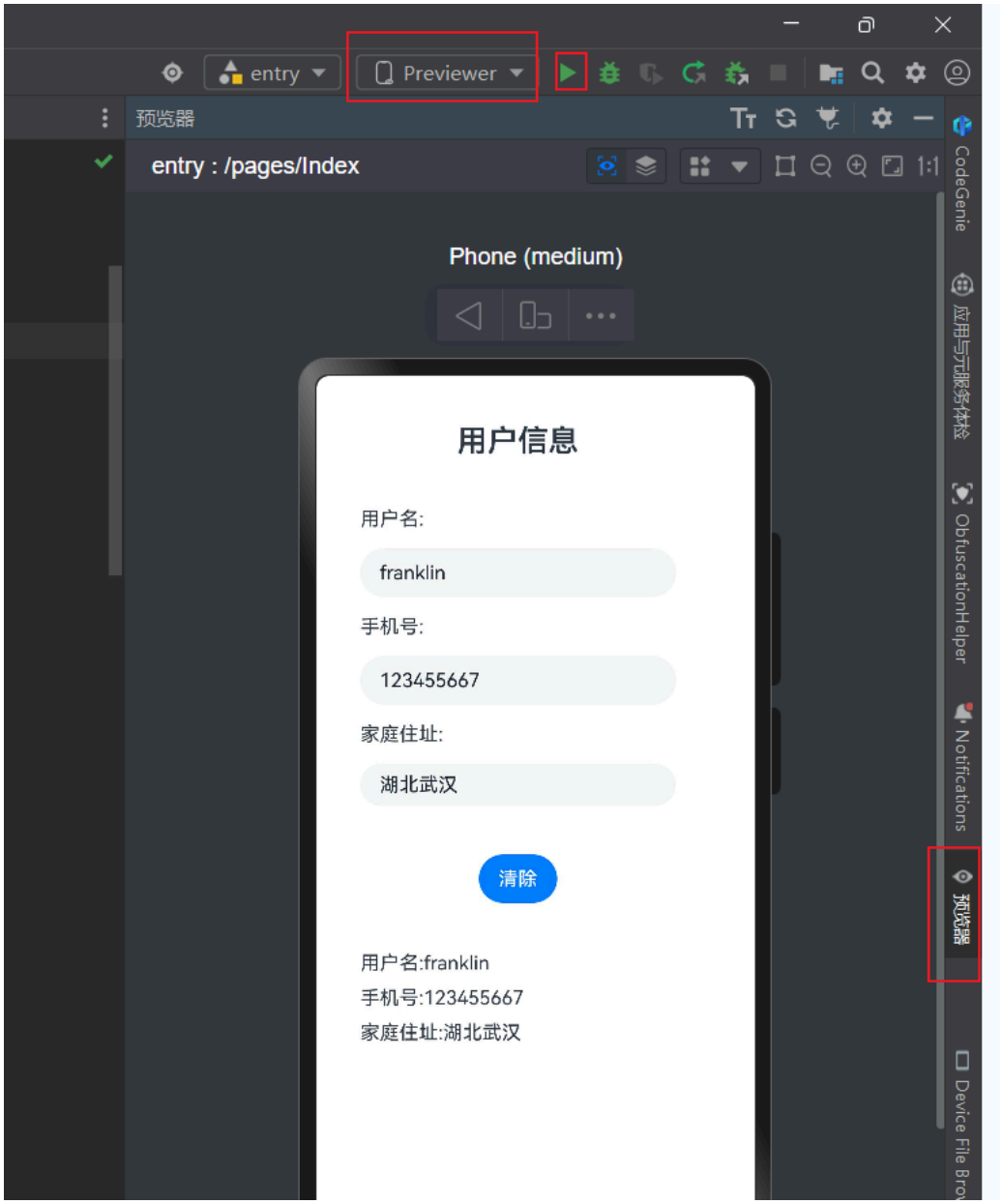
- 项目结构介绍



  - **entry**：Demo 主要代码存放目录
    - **src > main > ets**：用于存放ArkTS源码。
    - **src > main > ets > entryability**：应用/服务的入口。
    - **src > main > ets > entrybackupability**：应用提供扩展的备份恢复能力。
    - **src > main > ets > pages**：应用/服务包含的页面。

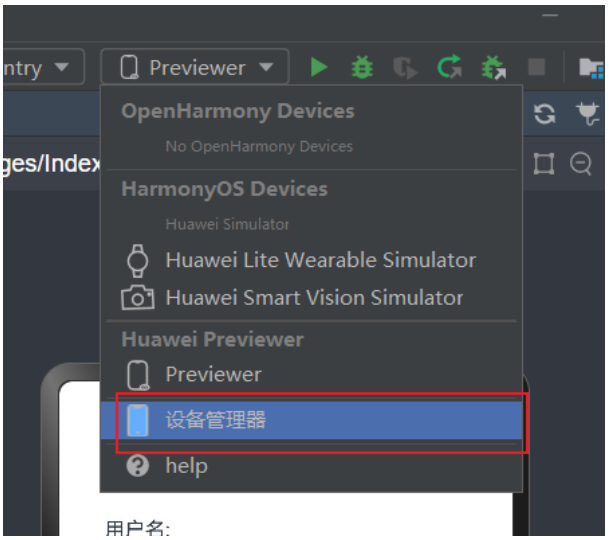
- **src > main > resources**：用于存放应用/服务所用到的资源文件，如图形、多媒体、字符串、布局文件等。关于资源文件，详见资源分类与访问。
- **src > main > module.json5**：模块配置文件。主要包含HAP包的配置信息、应用/服务在具体设备上的配置信息以及应用/服务的全局配置信息。具体的配置文件说明，详见module.json5配置文件。
- **build-profile.json5**：当前的模块信息 、编译信息配置项，包括buildOption、targets配置等。
- **hvigorfile.ts**：模块级编译构建任务脚本。
- **obfuscation-rules.txt**：混淆规则文件。混淆开启后，在使用Release模式进行编译时，会对代码进行编译、混淆及压缩处理，保护代码资产。详见开启代码混淆。
- **oh-package.json5**：用来描述包名、版本、入口文件（类型声明文件）和依赖项等信息。

项目实时预览

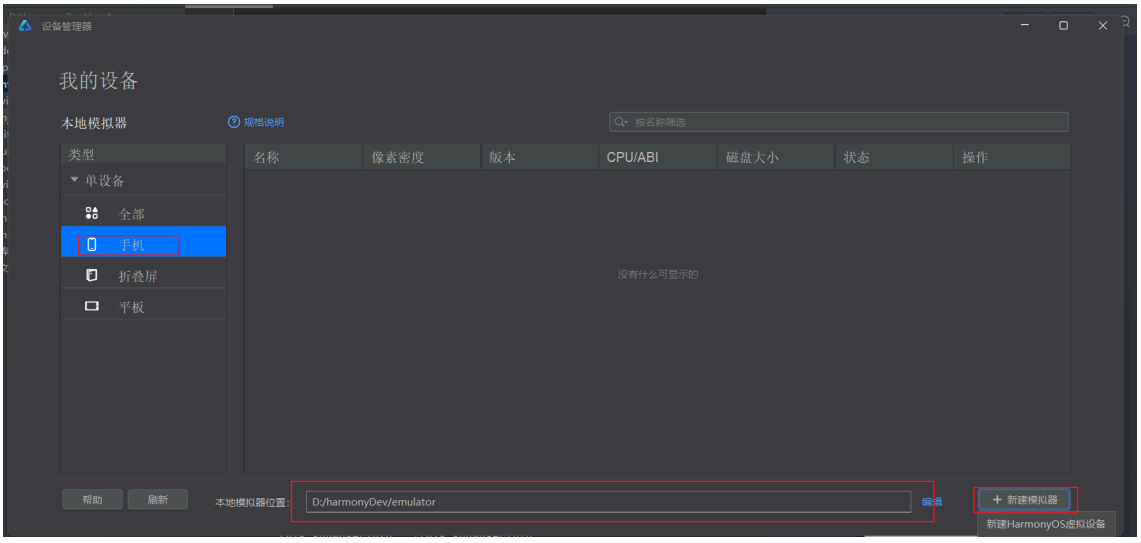


模拟器运行

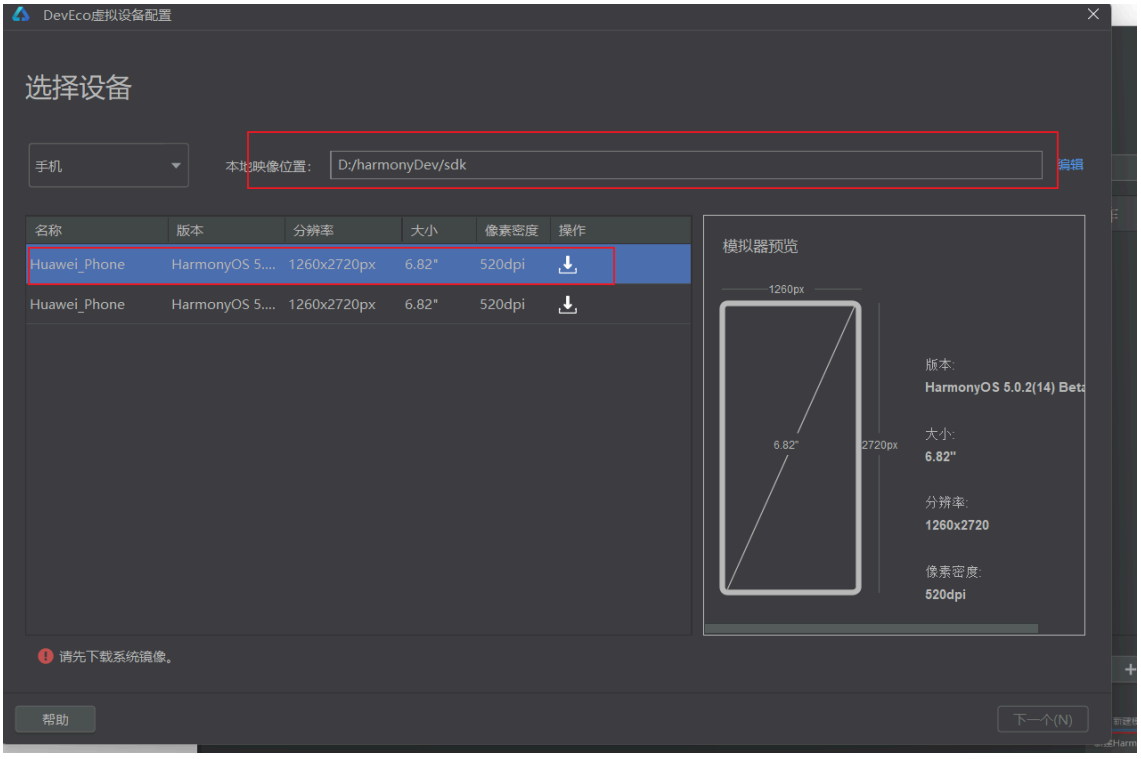
1、选择模拟器安装位置

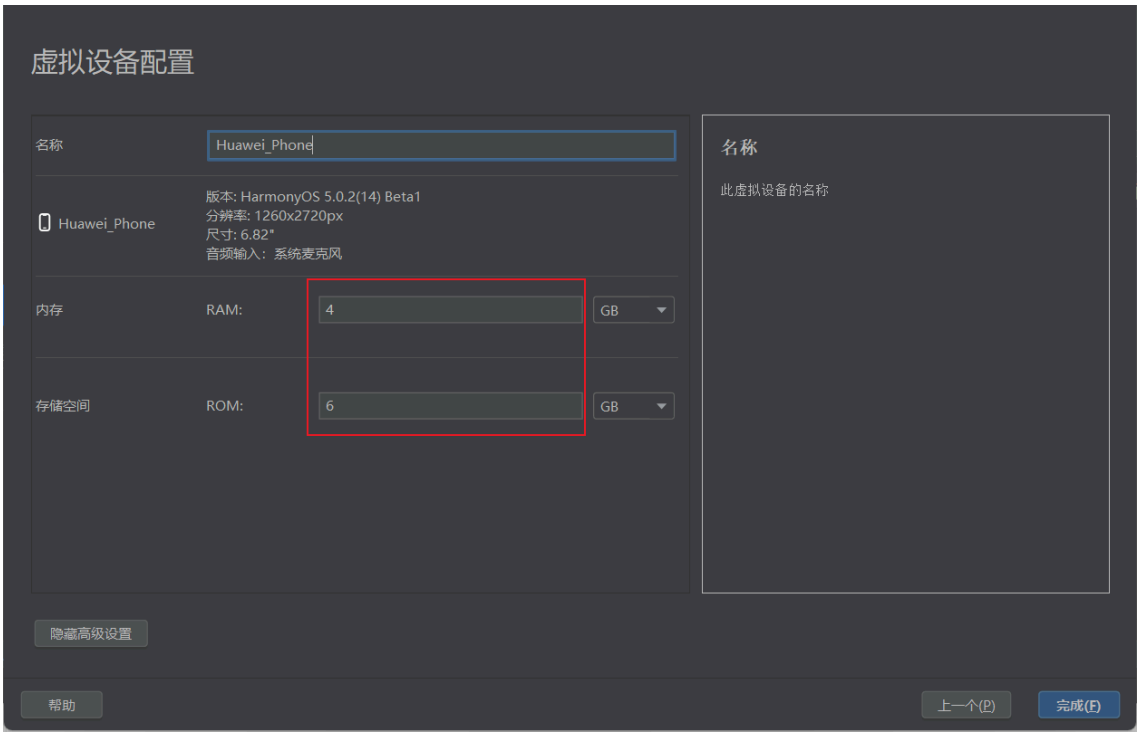


2、选择模拟类型，下载模拟器



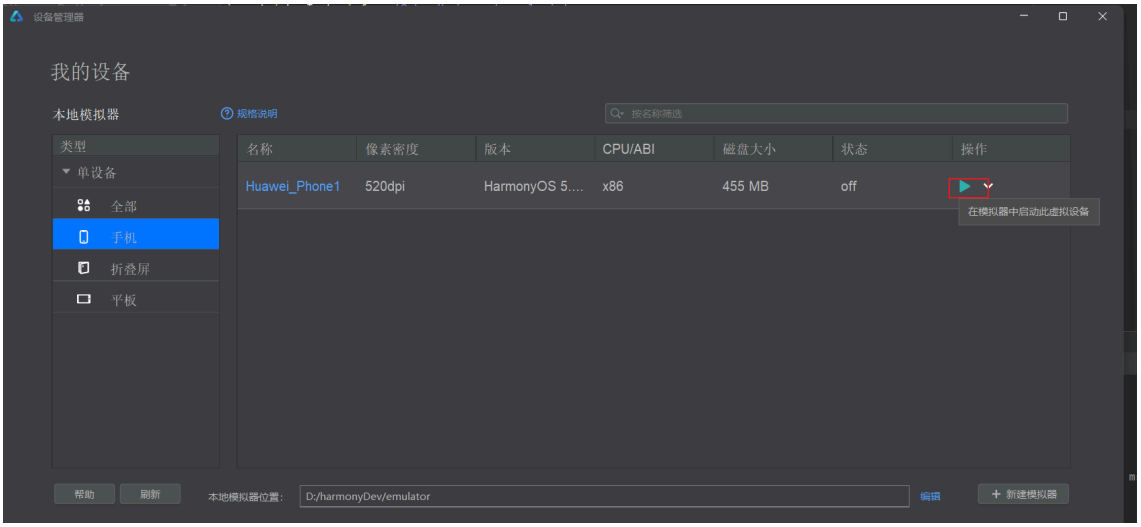
3、选择 SDK 安装位置



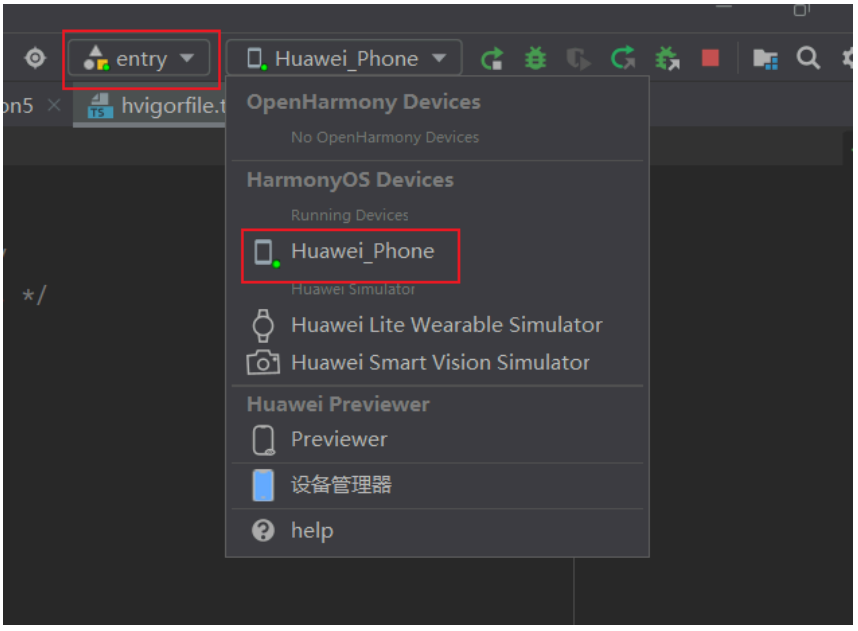


根据自己的电脑的配置选择内存和存储空间，点击完成。模拟器新建完成

返回模拟器列表页面，看到新建的模拟器，点击运行



会在预览下拉选项看到新建的模拟器



选择模拟器，点击运行，app的效果在模拟器上可以看到





本地打包流程

HAP、App、HAR 的关系

App是个上架概念，多个HAP打包一起上架。
HAP是可以独立运行、分发的，HAP不是复用的，复用的应该是HAR。
HAR是静态共享包，每个模块依赖的话都会打包到HAP里

APP包与HAP包的概念：

一个HarmonyOS工程的构建产物为APP包，APP包用于应用/服务发布上架应用市场。

HAP是应用/服务可以独立运行在设备中的形态，也是应用安装的基本单位，在DevEco Studio工程目录中，一个HAP对应一个Module。应用打包时，每个Module生成一个.hap文件。

两者之间的关联：

应用如果包含多个Module，在应用市场上架时，会将多个.hap文件打包成一个.app文件（称为Bundle），但在云端分发和端侧安装时，仍然是以HAP为基本单位。

为了能够正常分发和安装应用，需要保证一个应用安装到设备时，Module的名称、Ability的名称不重复，并且只有一个Entry类型的Module与目标设备相对应。

app里编的hap包一般会比正常编hap包小，编译app包默认是非debug的，app包里的hap包是混淆过的，不包含调试信息。

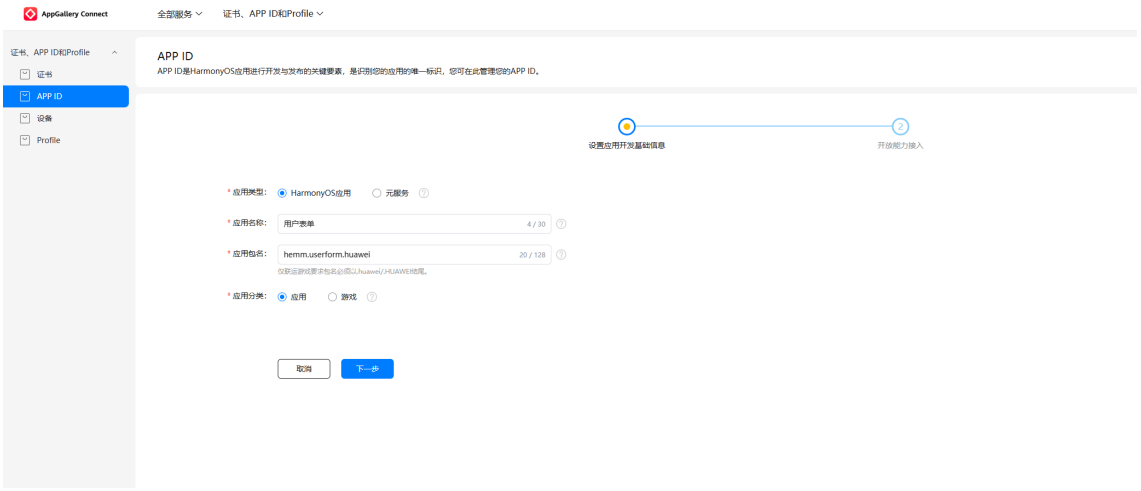
app是个上架概念，多个hap打包一起上架；

```
20
21  hap是可以独立运行、分发的，hap不是复用的，复用的应该是har；har是静态共享包，每个模块依赖的话都会打包到hap里
```

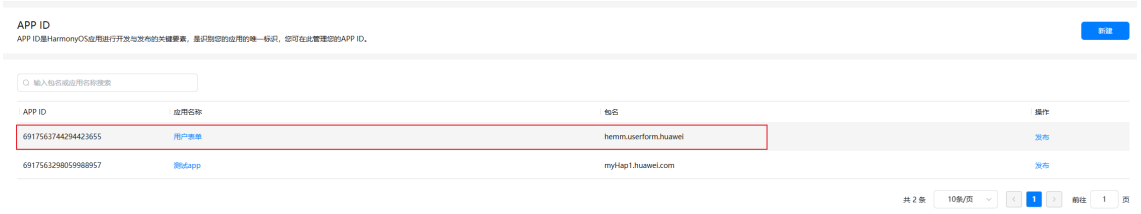
在AppGallery Connect 中创建项目流程

[创建应用流程]

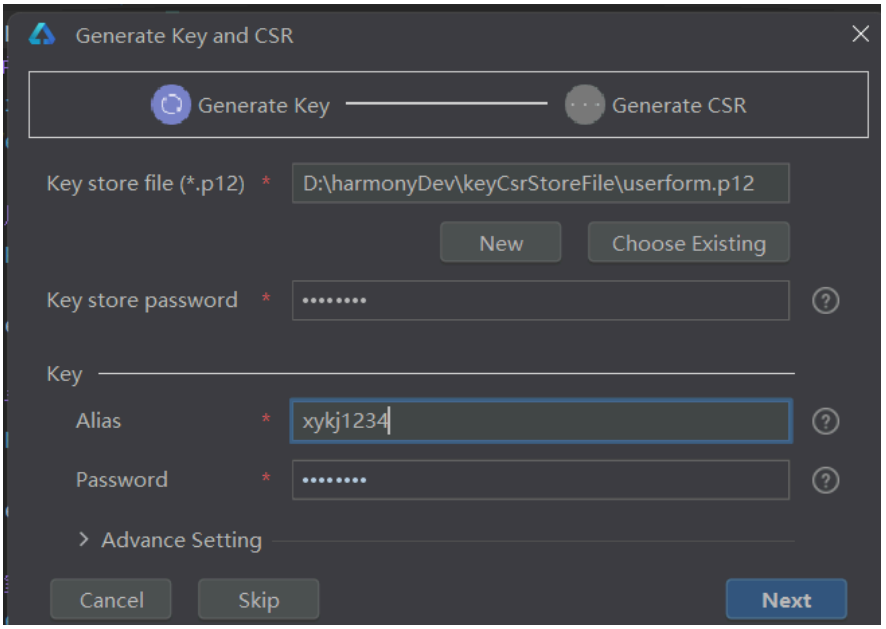
创建项目之后，添加对应的应用到项目中



项目创建完毕，项目列表可以看到刚创建的项目，以及项目中包含的应用



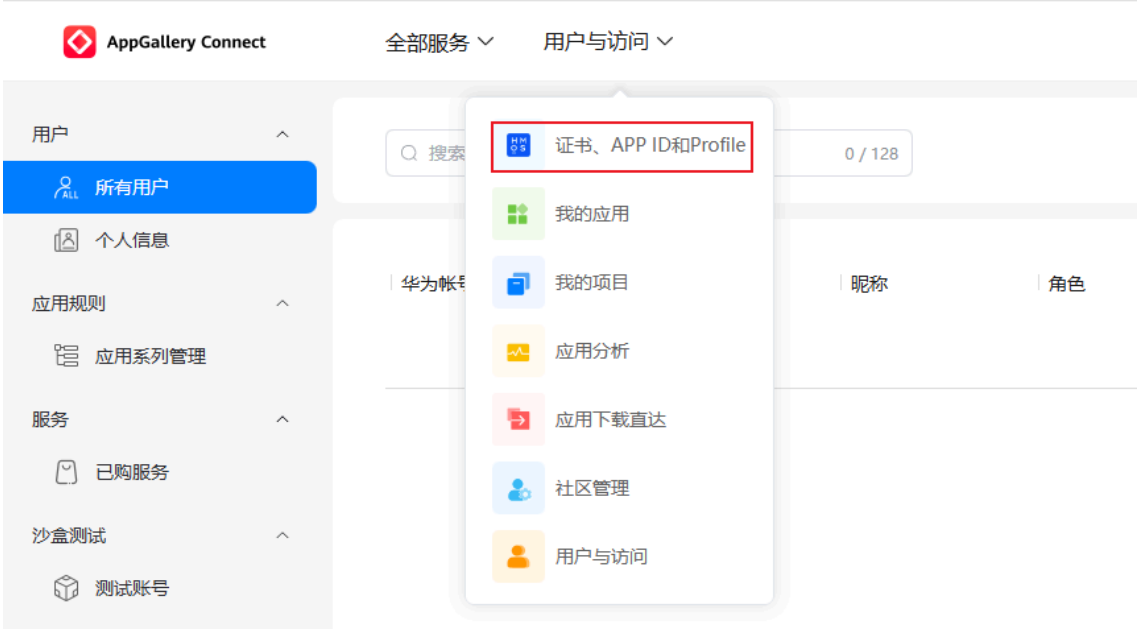
1、生成密钥和证书请求文件



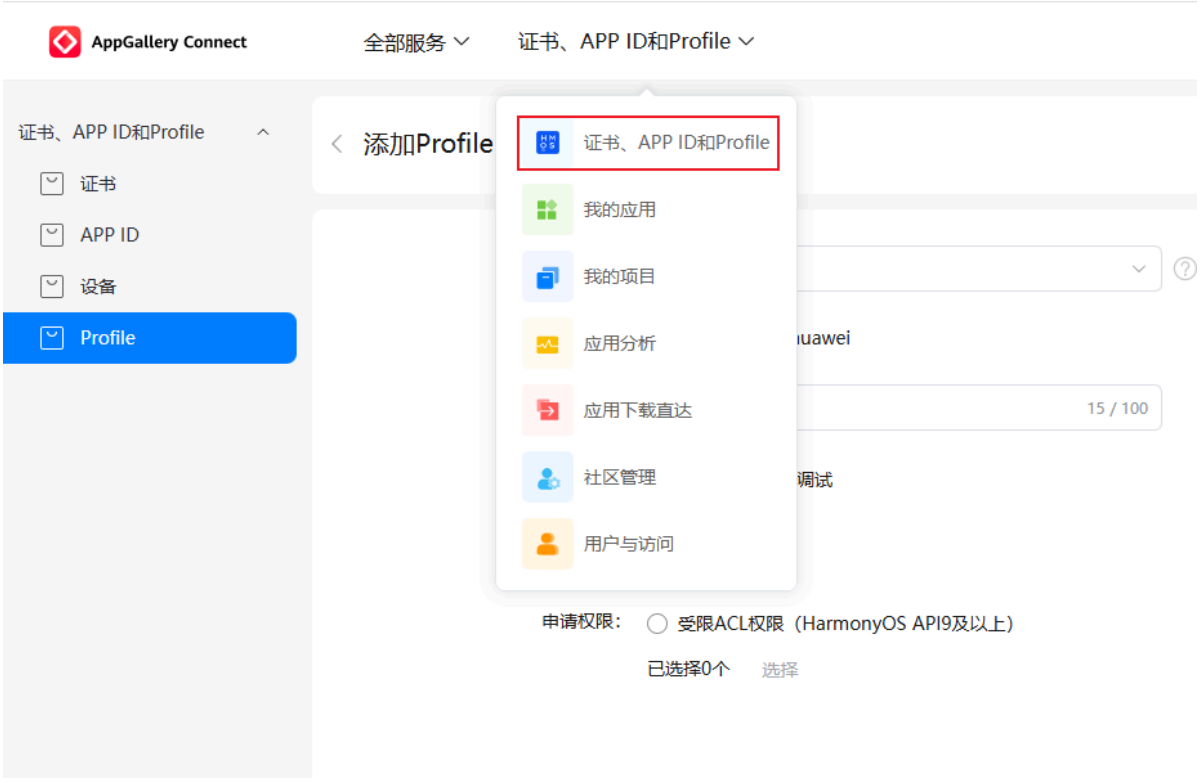
2、申请发布证书

1. 登录AppGallery Connect，选择“用户与访问”。
2. 选择个人信息，不要选择所有用户，所有用户对应的团队开发
3. 下拉选择 **证书、APPID 和Profile**

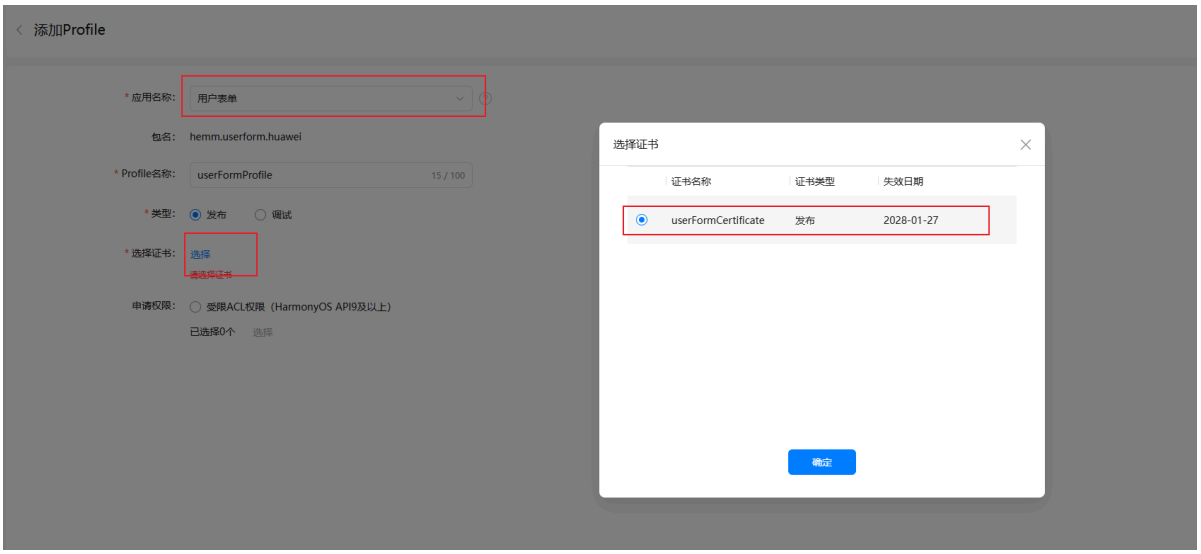
![image-20250127174634564](D:\谐云\谐云项目信息\鸿蒙开发\鸿蒙开发流程.assets\image-20250127174634564.png



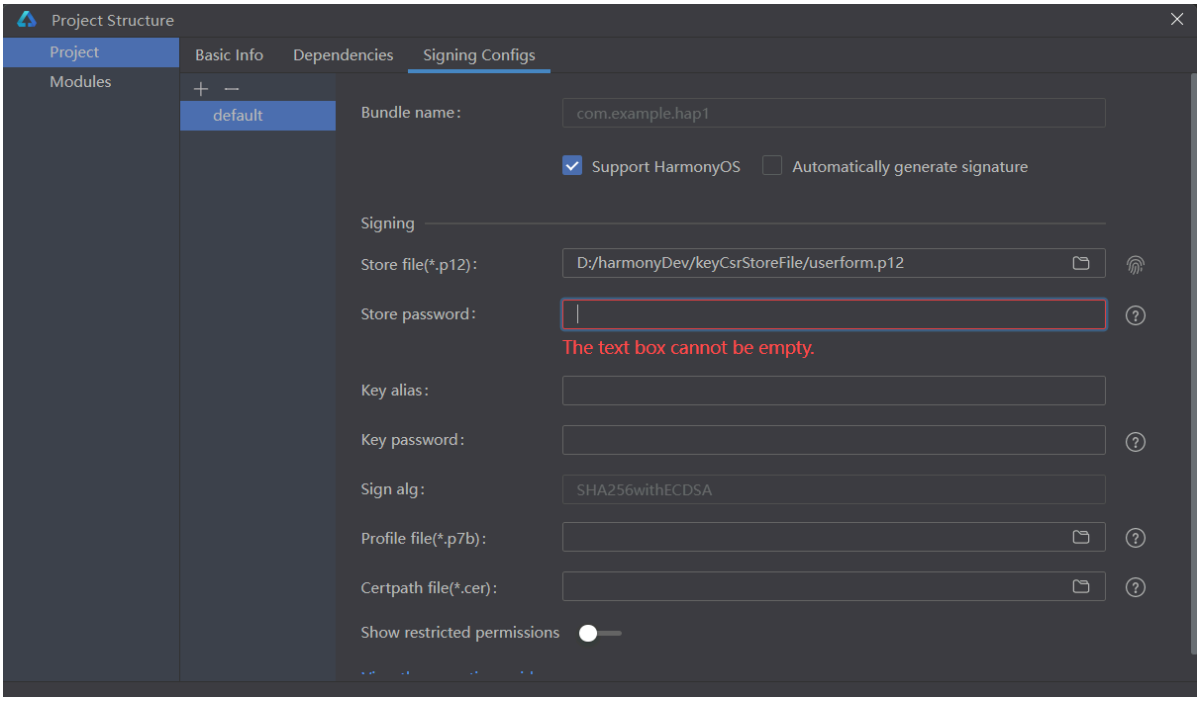
3、申请发布Profile



选择项目中的应用名称、上个步骤中生成的发布证书



4、配置签名信息

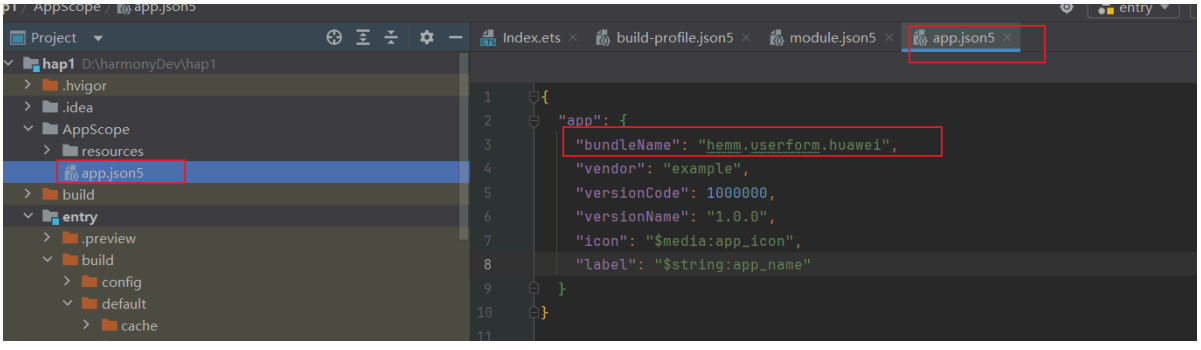


需要用的信息和各种证书在以上的步骤中已经生成，如实填写

5、编译打包

要注意 Bundle name 名字要和在 线上创建的应用包名一致，否则本地会编译失败

如果不一致可以在本地修改成和线上一致



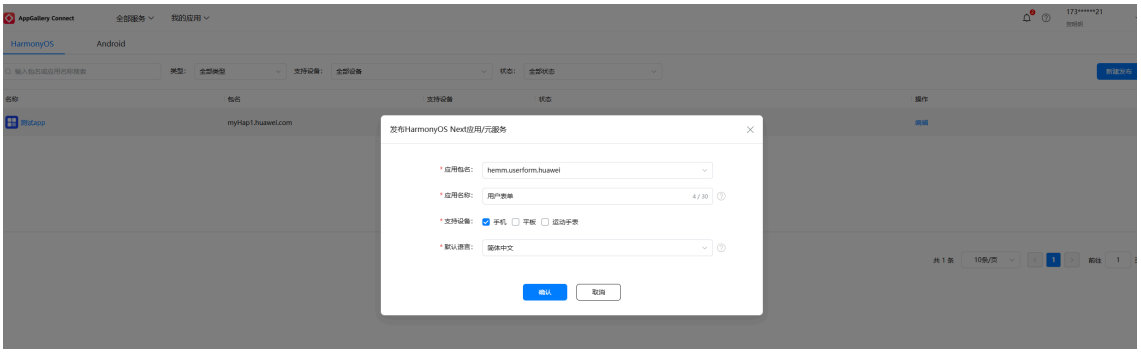
本地构建步骤

1. 打开DevEco Studio，菜单选择“Build > Build Hap(s)/APP(s) > Build APP(s)”。
2. 等待编译构建签名的HarmonyOS应用/元服务，编译完成后，可获取用于上架的软件包。

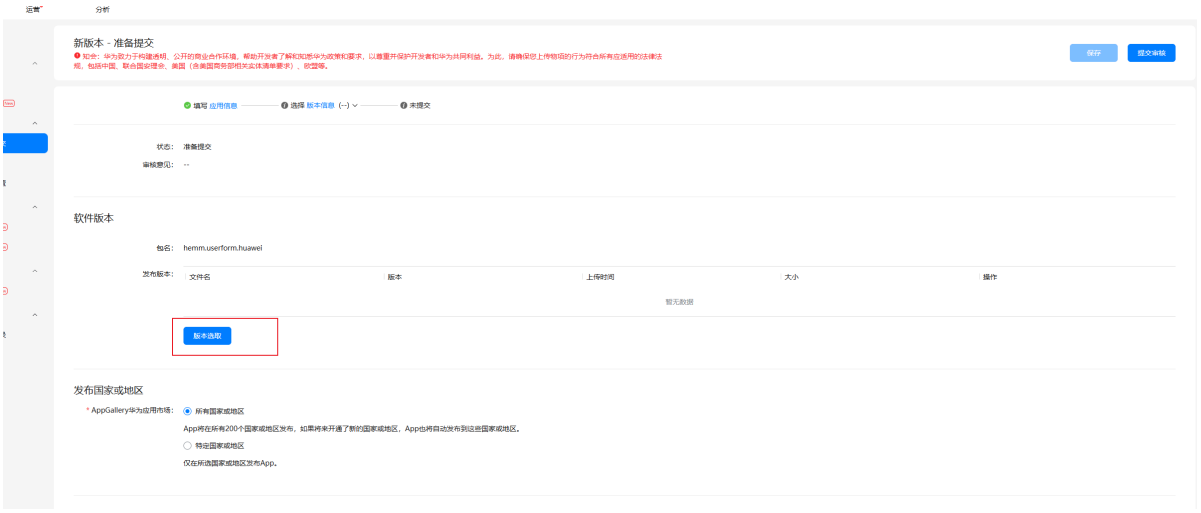
- API 9以前的应用/元服务软件包获取路径：工程目录build > outputs > app > release目录。
- API 9应用/元服务软件包获取路径：工程目录build > outputs > default目录。

线上发布流程

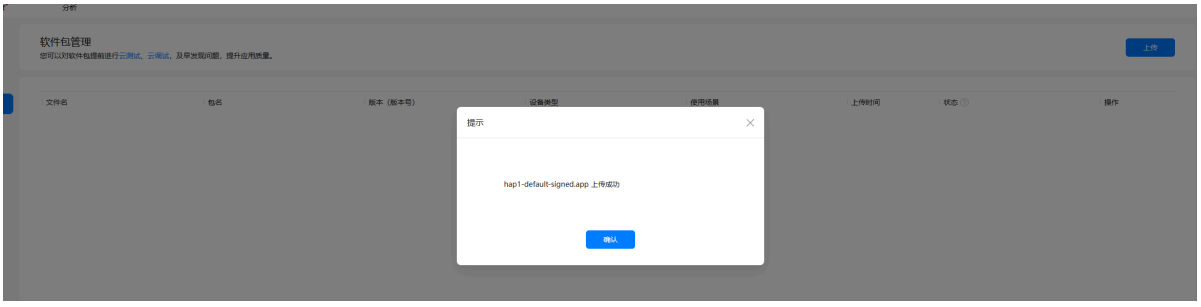
1. 登录AppGallery Connect，选择“我的应用”。
2. 在应用列表首页中点击“HarmonyOS应用”页签。



本地编译好的应用会在列表中显示，没有显示，多刷新几次页面，出现弹窗 点击确认，填写应用相关信息即可。点击下一步选择 ，继续填写信息。

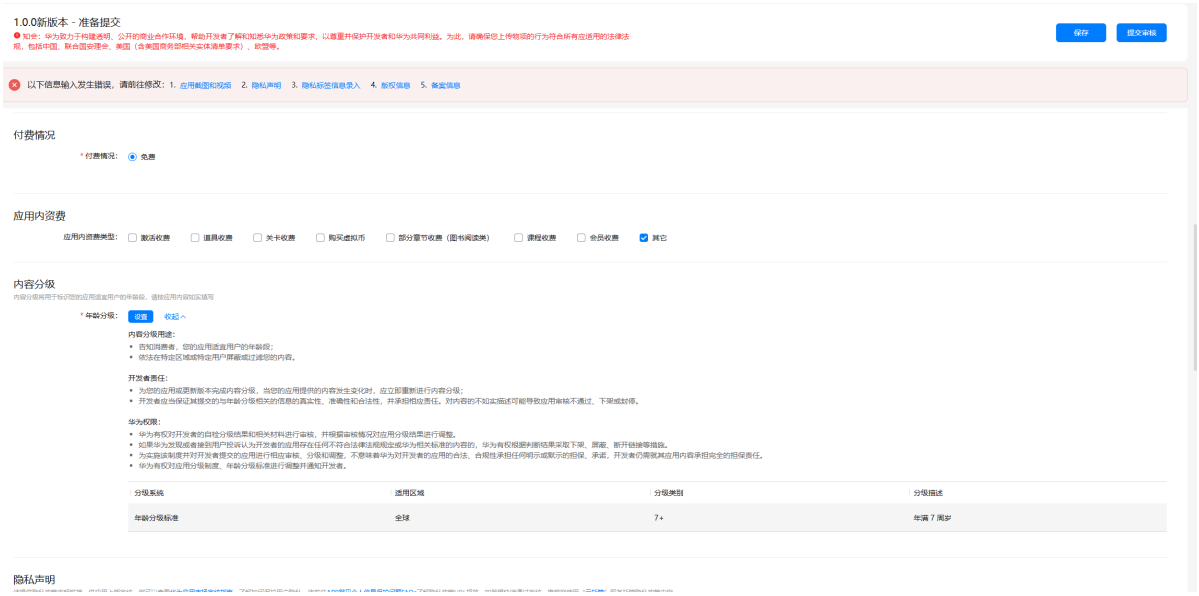


版本选择就是上传软件包，选择本地打包编译的app 包



上传成功

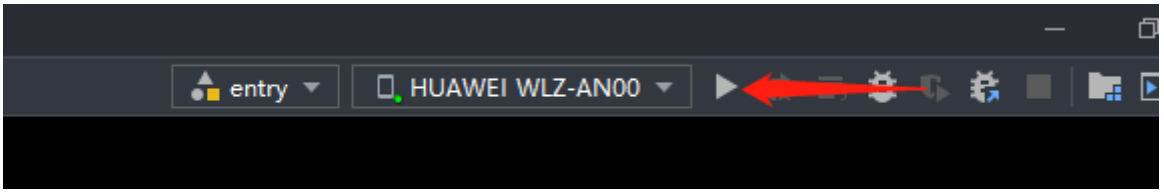
此页面要填写的信息巨长，未填写



真机调试流程

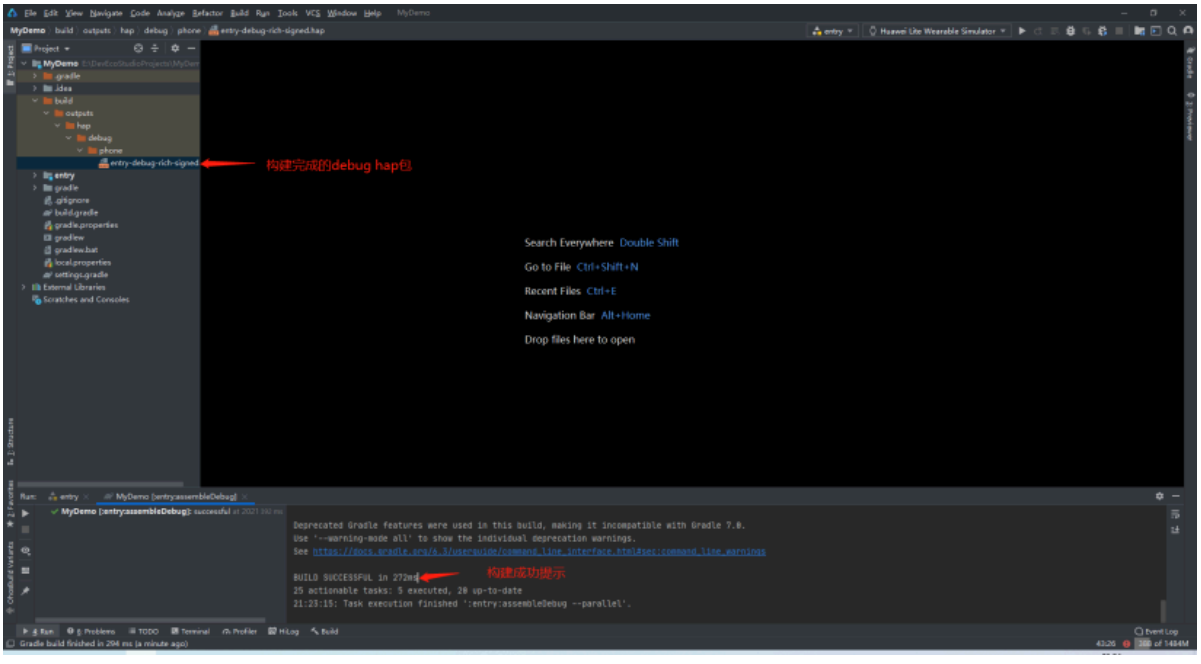
直接通过ide安装应用

该方法只用于开发调试，在连接手机后，点击该运行按钮就行。



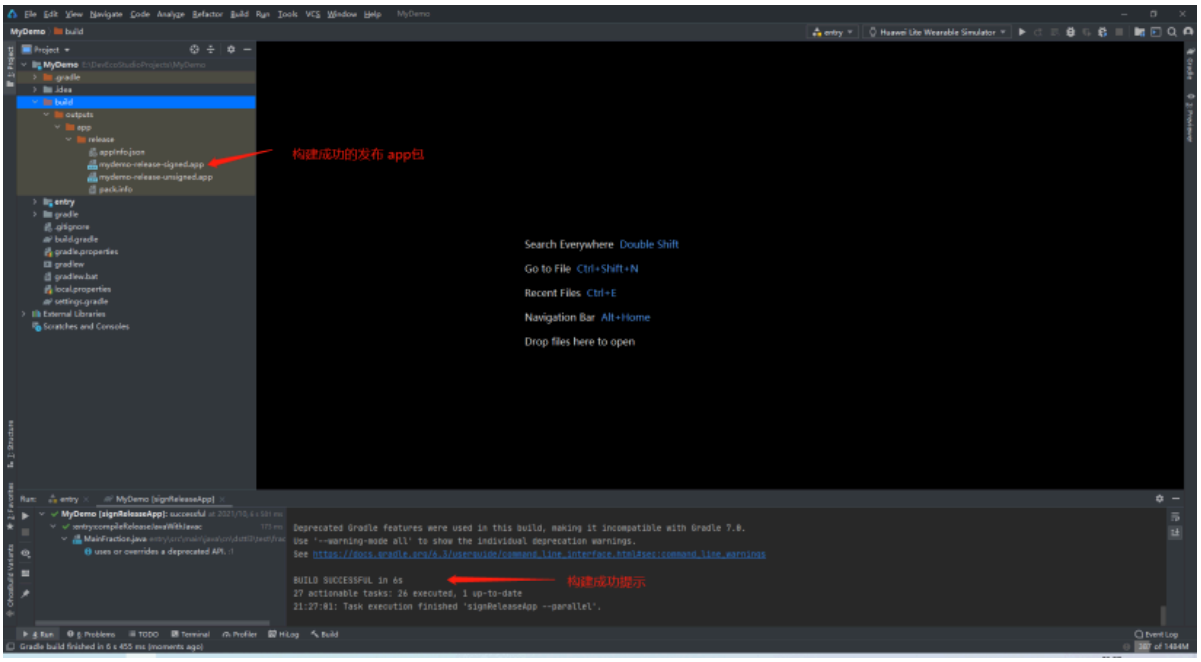
打包debug版本包

打开DevEco Studio 进入Build > Build Hap(s)/APP(s) > Build Hap(s)



打包Release版本包

打开DevEco Studio 进入Build > Build Hap(s)/APP(s) > Build APP(s)



应用安装

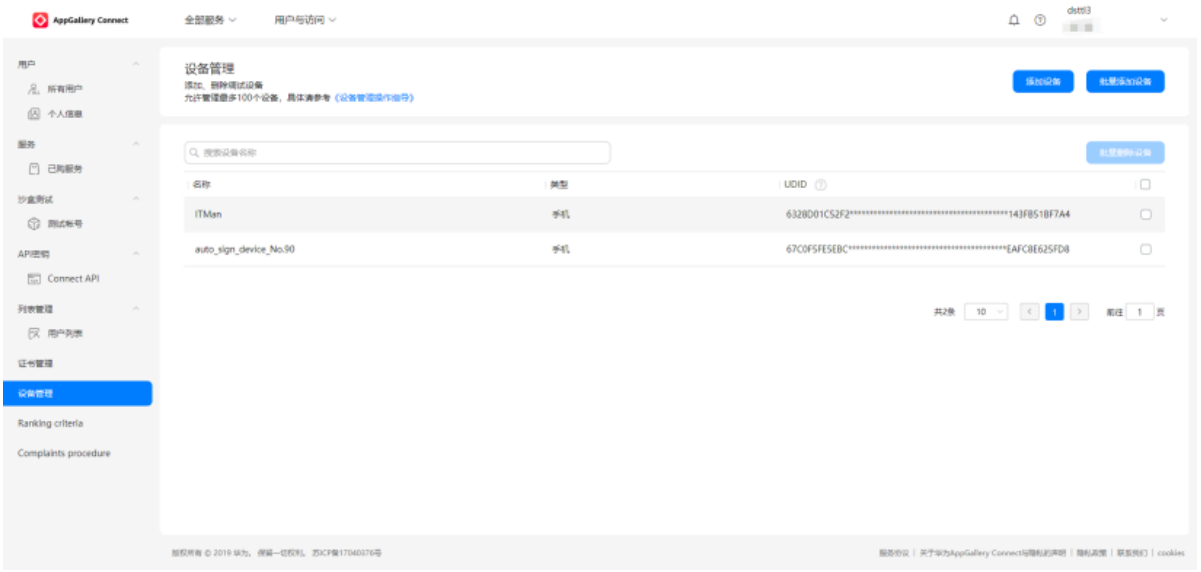
应用安装分为两种，一种是debug版本安装，一种是通过应用商店安装。

- debug版本安装

1.确认设备是否已经被添加到测试设备

查看方式

登录华为AppGallery Connect -> 用户与访问 -> 设备管理



查看设备是否在列表中。

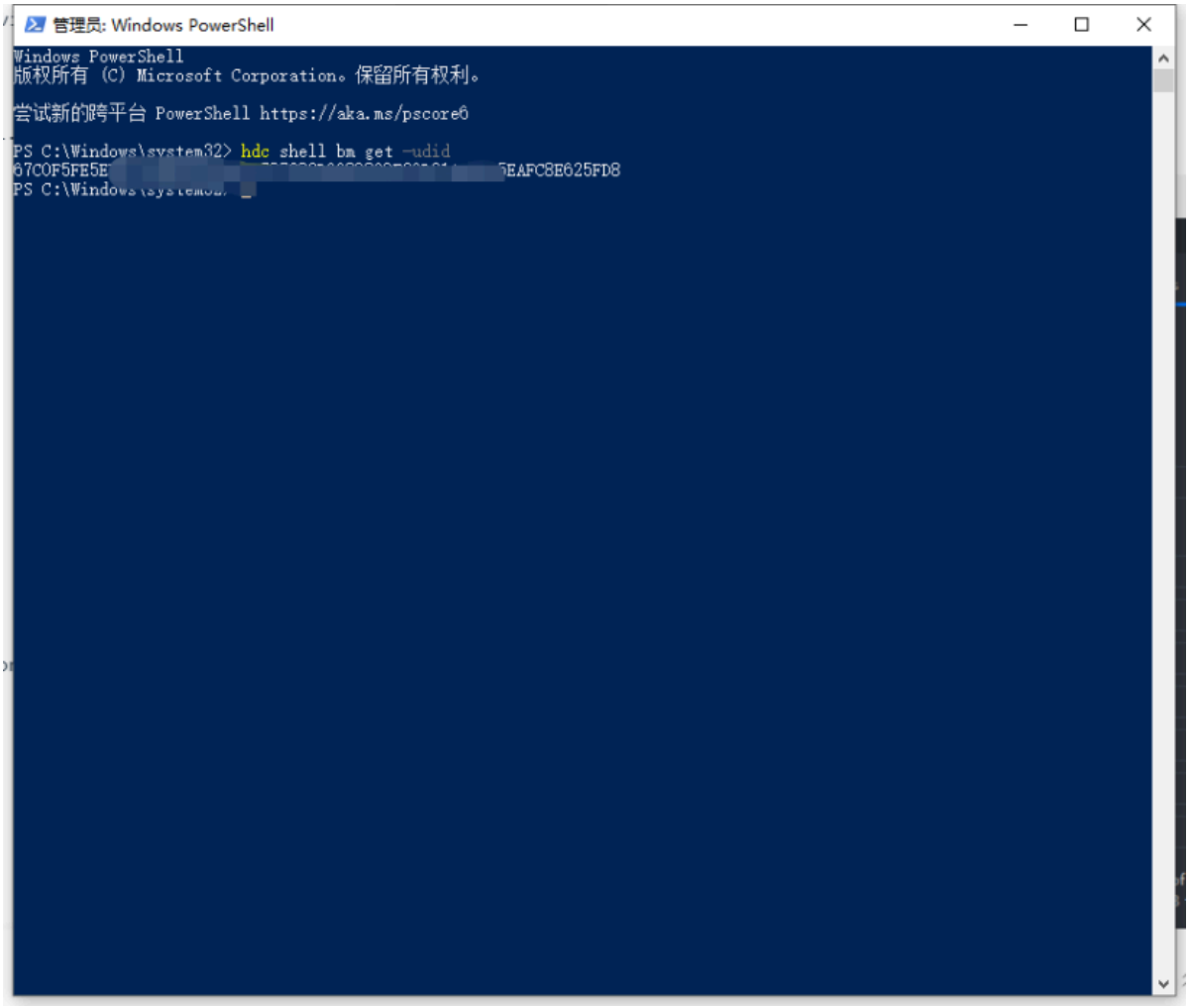
设备UDID可以通过hdc工具查询(附件附带，或者在鸿蒙SDK中找到。)

使用数据线将手机连接到电脑，打开cmd命令

查询命令：

复制

```
hdc shell bm get -udid
1.
```



如果应用没有添加，需要添加设备后，重新生成调试证书，然后替换先前的证书。

- 安装应用

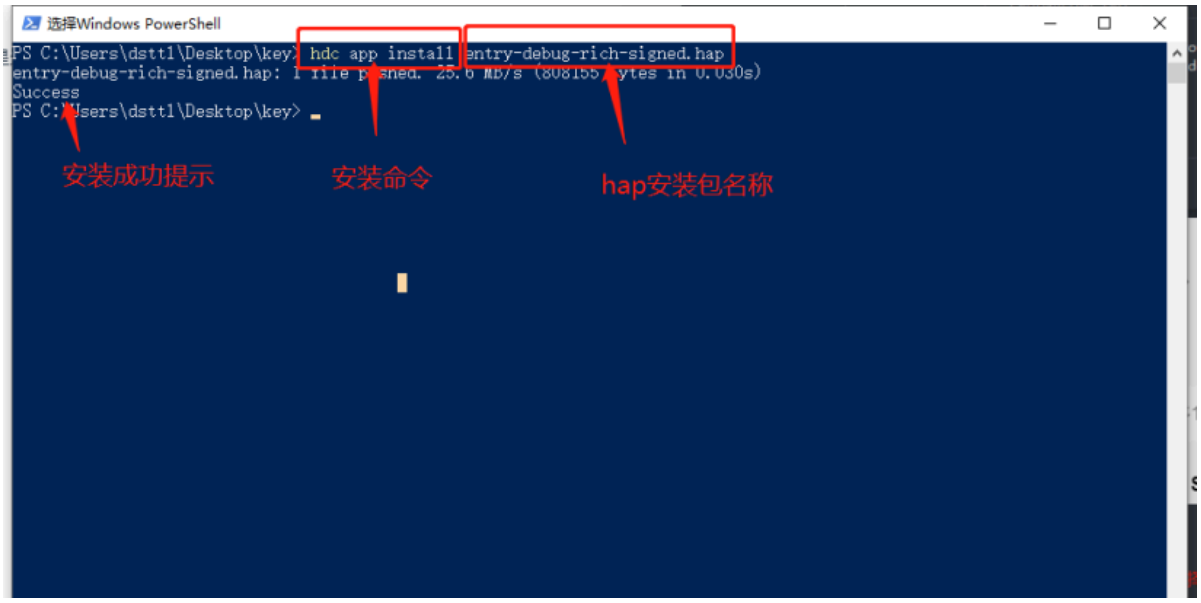
因为鸿蒙目前并不像安卓，支持安装包直接在手机上安装，所以如果是多台设备或多个测试人员，需要通过hdc工具来进行安装

首先需要确认上述步骤是否已经完成

打开CMD，进入构建完成的debug hap包目录，输入以下命令：

复制

```
hdc app install entry-debug-rich-signed.hap
1.
```



到此，应用就成功安装到手机上了